

**ÇANKIRI KARATEKİN ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



**AI-Vision3D: Gerçek Zamanlı Nesne Tanıma ve 3B Görsel Destekli
Bilgilendirme Sistemi**

BİTİRME PROJESİ

Mehmet DURMAZ

2024-2025 BAHAR DÖNEMİ

**ÇANKIRI KARATEKİN ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**

**AI-Vision3D: Gerçek Zamanlı Nesne Tanıma ve 3B Görsel Destekli
Bilgilendirme Sistemi**

BİTİRME PROJESİ

Mehmet DURMAZ

2024-2025 BAHAR DÖNEMİ

ÖNSÖZ

Bu bitirme projesi, yapay zeka ve üç boyutlu görselleştirme teknolojilerini bir araya getirerek nesne tespiti yapılan bir sahnede, kullanıcıya anlık ve etkili bir görsel destek sunmayı hedefleyen bir sistem geliştirme amacıyla hazırlanmıştır. Python diliyle oluşturulan grafiksel arayüz ve derin öğrenme modellerinin entegrasyonu sayesinde, gerçek zamanlı olarak tanınan nesnelerin üç boyutlu temsilleri kullanıcıya sunulmuş, bu sayede görsel anlamda bilgilendirici ve öğretici bir deneyim sağlanmıştır.

Bu projenin hayata geçmesinde katkısı olan pek çok değerli insan vardır. Öncelikle, eğitim hayatım boyunca her daim yanımda olan, dualarıyla ve desteğiyle bana güç veren canım annem **Kadriye Durmaz** ve sevgili babam **Ahmet Durmaz**'a en içten teşekkürlerimi sunuyorum. Aynı şekilde, hayatımda hep yanımda olup bana moral veren kardeşlerim **Asude Kevser Durmaz**, **Yusuf Erdem Durmaz**, **Fatma Nida Durmaz** ve **Elif Ebrar Durmaz**'a sevgilerimi sunarım.

Bu süreçte fikirleriyle, sohbetleriyle ve dostluklarıyla bana destek olan arkadaşlarım **İmran Ovacı**, **İclal Akkaya**, **Muhammet Enes Aydın** ve **Berk Egeli** 'ye teşekkür ederim. Akademik sürecimde bana yol gösteren, desteğini esirgemeyen değerli hocam **Doktor Öğretim Üyesi Seda Şahin** de saygı ve teşekkürlerimi sunarım.

Bu proje, yalnızca teknik bir çalışma değil; aynı zamanda birlikte üretmenin, öğrenmenin ve dayanışmanın bir ürünü olmuştur.

Mehmet DURMAZ
Çankırı 2025

İÇİNDEKİLER

	Sayfa No
ÖNSÖZ.....	II
İÇİNDEKİLER.....	III
ÖZET.....	IV
1. GENEL BİLGİLER.....	1
1.1. Giriş.....	2
1.2. Amaç ve Hedefler.....	3
1.3. Literatüre Katkı.....	4
1.4. Tez Çalışması Akış Şeması	5
2. LİTERATÜR TARAMASI	6
3. MATERYAL VE METOT.....	7
3.1. Veri Seti Açıklaması	8
3.2. Makine Öğrenmesi Algoritmaları.....	9
3.3.	10
3.4.....	11
4. BULGULAR VE TARTIŞMA.....	12
4.1.	13
4.2.	14
5. SONUÇ VE ÖNERİLER	15
5.1.	16
5.2.	17
6. KAYNAKLAR.....	18
7. EKLER (varsa).....	19

ÖZET

Bu çalışmada, gerçek zamanlı nesne tespiti ve 3 boyutlu görsel bilgilendirme işlevlerini bir arada sunan, Python tabanlı etkileşimli bir kullanıcı arayüzü geliştirilmiştir. Projede, derin öğrenmeye dayalı YOLOv8 (You Only Look Once) mimarisi kullanılarak nesne algılama işlemleri gerçekleştirilmiş; ardından tespit edilen nesnelere karşılık gelen üç boyutlu modeller veya temsili görseller, kullanıcı arayüzünde dinamik olarak sunulmuştur. Böylece kullanıcıya hem metinsel hem de görsel olarak desteklenmiş, açıklayıcı ve sezgisel bir etkileşim ortamı sağlanmıştır.

Nesne tespiti için kullanılan YOLOv8 algoritması, hızlı ve yüksek doğruluklu tahmin kapasitesiyle gerçek zamanlı uygulamalarda sıklıkla tercih edilen evrimsel bir modeldir (Jocher et al., 2023). Modelin eğitimi ve çalıştırılması sırasında PyTorch derin öğrenme kütüphanesi kullanılmıştır (Paszke et al., 2019). Arayüz, Python'un modern GUI araçlarından biri olan PySide6 ile geliştirilmiş; bu sayede kullanıcı dostu ve modüler bir sistem mimarisi elde edilmiştir. Görselleştirme katmanında ise 3B modellerin entegrasyonu için uygun dosya formatları (.obj, .glb) desteklenerek görsel zenginlik sağlanmıştır.

Sistem, özellikle eğitim teknolojileri, güvenlik uygulamaları ve tanıtım sistemleri gibi alanlarda potansiyel kullanım senaryolarına sahiptir. Proje, bilgisayarla görüş (computer vision), derin öğrenme (deep learning) ve insan-bilgisayar etkileşimi (HCI) disiplinlerinin kesişiminde konumlanmakta olup, gelecekte artırılmış gerçeklik (AR) ve doğal dil işleme (NLP) teknolojileriyle de genişletilmeye açıktır.

1. GENEL BİLGİLER

1.1. Giriş

Günümüzde yapay zekâ, bilgisayarla görü ve artırılmış gerçeklik (AR) teknolojileri birçok sektörde hızlı bir şekilde yaygınlaşmakta ve dijital dönüşümün temelini oluşturmaktadır. Bu teknolojiler sayesinde, gerçek dünya verileri anlık olarak analiz edilebilmekte ve kullanıcıya çeşitli şekillerde sunulabilmektedir. Özellikle derin öğrenmeye dayalı nesne tespiti algoritmaları sayesinde görüntü işleme alanında önemli ilerlemeler kaydedilmiş, bu algoritmalar; güvenlik sistemlerinden sağlık uygulamalarına, sanayi otomasyonundan eğitim teknolojilerine kadar birçok alanda etkin biçimde kullanılmaya başlanmıştır.

Bu çalışmada, Python programlama dili kullanılarak geliştirilen bir grafiksel kullanıcı arayüzü (GUI) ile gerçek zamanlı nesne tespiti yapılmakta ve bu nesnelere ait üç boyutlu (3B) görseller sistem arayüzünde kullanıcıya sunulmaktadır. YOLOv8 (You Only Look Once) algoritmasıyla yapılan tespitler, kullanıcıya görsel açıdan zengin bir bilgilendirme sağlamak amacıyla 3B nesne temsilleri ile desteklenmiştir. Bu sayede sistem, yalnızca bir nesnenin ne olduğunu tespit etmekle kalmamakta, aynı zamanda kullanıcıya daha anlaşılır ve etkileşimli bir deneyim sunmaktadır.

Projenin temel amacı, bilgisayarla görü teknolojilerinin sunduğu olanakları, sezgisel ve kullanıcı dostu bir arayüzle birleştirerek; kullanıcıların yapay zekâ destekli sistemlerle kolayca etkileşime geçebileceği bir ortam oluşturmaktır. Böylece bu çalışma; eğitim, savunma, tanıtım ve rehberlik gibi alanlarda uygulanabilir, esnek ve modüler bir sistem sunmaktadır.

1.2 Amaç ve Hedefler

Bu bitirme çalışmasının temel amacı, gerçek zamanlı nesne tespiti ile bu nesnelere ait üç boyutlu (3B) görsellerin eşzamanlı olarak kullanıcıya sunulabildiği, etkileşimli, görsel açıdan zengin ve kullanıcı odaklı bir sistem geliştirmektir. Sistem, Python programlama dili kullanılarak tamamen özgün bir şekilde tasarlanmış olup, sezgisel bir grafiksel kullanıcı arayüzü (GUI) üzerinden çalışmaktadır. Derin öğrenme tabanlı nesne tanıma algoritmalarına dayanan bu sistem, tespit edilen nesnelerin 3B görsel temsillerini arayüzde anında kullanıcıya sunarak, hem bilgilendirici hem de etkileşimli bir deneyim sağlamayı hedeflemektedir. Bu bağlamda, bilgisayarla görü, derin öğrenme ve grafiksel arayüz tasarımı teknolojilerinin entegrasyonu, çalışmanın temel taşlarını oluşturmaktadır.

Sistem, dışa bağımlı artırılmış gerçeklik (AR) kütüphaneleri veya hazır 3B modelleme araçları yerine, geliştirici tarafından oluşturulan özel modellemeler ve yazılım altyapısı kullanılarak inşa edilmiştir. Bu yaklaşım, sistemin bağımsız, özelleştirilebilir ve farklı uygulama alanlarına uyarlanabilir bir yapıda olmasını sağlamıştır. Geliştirilen yazılım, özellikle eğitim, tanıtım, endüstriyel otomasyon, güvenlik ve sağlık gibi çeşitli sektörlerde nesne tanıma ve görselleştirme ihtiyaçlarını karşılayabilecek modüler bir çözüm sunmaktadır. Kullanıcı arayüzünün sade, hızlı ve etkileşimli tasarımı, sistemin farklı platformlara (masaüstü, mobil veya gömülü sistemler) entegrasyonunu kolaylaştırarak, geniş bir kullanıcı kitlesine hitap etmesini mümkün kılmaktadır.

Çalışmada kullanılan derin öğrenme modelleri, nesne tespiti için modern konvolüsyonel sinir ağları (CNN) tabanlı algoritmalar (örneğin, YOLO veya Faster R-CNN) üzerine inşa edilmiştir. Bu modeller, gerçek zamanlı performans sağlamak amacıyla optimize edilmiş ve düşük donanım gereksinimleriyle çalışabilecek şekilde uyarlanmıştır. Ayrıca, 3B

görselleştirme için OpenGL veya benzeri grafik kütüphaneleri kullanılarak, tespit edilen nesnelerin detaylı ve dinamik temsilleri oluşturulmuştur. Bu görselleştirmeler, kullanıcıların nesneleri farklı açılardan inceleyebilmesine ve sistemle etkileşime geçebilmesine olanak tanımaktadır.

Sistemin modüler yapısı, farklı nesne sınıflarının tanınması ve görselleştirilmesi için yeni modellerin kolayca entegre edilebilmesine olanak sağlar. Bu özellik, sistemin gelecekteki geliştirme süreçlerinde esneklik sunarken, aynı zamanda akademik ve endüstriyel projelerde yeniden kullanılabilirliğini artırmaktadır. Kullanıcı arayüzü, Tkinter veya PyQt gibi Python tabanlı GUI kütüphaneleriyle geliştirilmiş olup, kullanıcı dostu bir deneyim sunmak için minimal ancak işlevsel bir tasarım felsefesi benimsenmiştir.

Bu doğrultuda, çalışmanın nihai hedefi; bilgisayarla görü, derin öğrenme ve 3B grafiksel arayüz tasarımı bir araya getirerek, gerçek zamanlı nesne tanıma ile zenginleştirilmiş, görsel olarak etkileyici ve etkileşimli bir deneyim sunabilen özgün bir sistem ortaya koymaktır. Sistem, hem teknik hem de kullanıcı odaklı yenilikleriyle, ilgili alanlarda fark yaratan bir çözüm olmayı amaçlamaktadır. Bu bitirme çalışması, önerilen sistemin tasarımından implementasyonuna kadar tüm aşamalarını detaylı bir şekilde ele alarak, hem akademik hem de pratik katkılar sunmayı hedeflemektedir.

1.3 Literatüre Katkı

Bu çalışma, görüntü işleme, yapay zekâ temelli nesne tanıma sistemleri ve üç boyutlu (3B) görsel sunumların entegrasyonunu bir arada ele alarak, literatürdeki benzer çalışmalara özgün bir katkı sunmayı amaçlamaktadır. Günümüzde nesne tespit sistemleri, genellikle sınıflandırma, işaretleme veya sınırlı görsel geri bildirim sağlama gibi işlevlerle kısıtlı kalmaktadır. Bu sistemler, tespit edilen nesnelere dair kullanıcıya derinlemesine, görsel açıdan zengin ve etkileşimli bilgi sunma konusunda yetersizdir (Krizhevsky et al., 2012). Önerilen sistem, tespit edilen nesnelerin yalnızca etiketlenmesiyle yetinmeyip, aynı zamanda bu nesnelere ait 3B modellerin grafiksel kullanıcı arayüzü (GUI) üzerinden eşzamanlı olarak görselleştirilmesini sağlayarak, literatürdeki bu boşluğu doldurmayı hedeflemektedir.

Çalışmada, nesne tanıma için derin öğrenme tabanlı algoritmalar, özellikle YOLOv5 (Jocher et al., 2021) ve Faster R-CNN (Ren et al., 2015) gibi konvolüsyonel sinir ağları (CNN) kullanılmıştır. Bu algoritmalar, yüksek doğruluk ve gerçek zamanlı performans sağlamak amacıyla optimize edilmiştir. 3B görselleştirme için ise OpenGL tabanlı özel bir render motoru geliştirilmiş, böylece nesnelerin dinamik ve detaylı temsilleri kullanıcıya sunulmuştur. Bu yaklaşım, artırılmış gerçeklik (AR) veya sanal gerçeklik (VR) gibi teknolojilere bağımlı olmadan, tamamen özgün bir görselleştirme altyapısı sunmaktadır. Sistemin modüler yapısı, farklı nesne sınıflarının tanınması ve yeni 3B modellerin kolayca entegre edilebilmesi için esneklik sağlamaktadır.

Proje kapsamında, yaygın olarak kullanılan hazır AR platformları (örneğin, ARKit veya ARCore) ve nesne tanıma servisleri (örneğin, Google Vision API) yerine, geliştirici tarafından oluşturulan bağımsız algoritmalar ve modeller tercih edilmiştir. Bu yaklaşım,

yazılımın dış kütüphanelere olan bağımlılığını azaltarak daha esnek, kişiselleştirilebilir ve maliyet etkin bir çözüm sunmuştur (Billinghurst et al., 2015). Nesne tanıma, görselleştirme ve kullanıcı arayüzü bileşenlerinin tek bir çatı altında entegre edilmesi, sistemin hem akademik araştırmalarda hem de endüstriyel uygulamalarda kullanılabilirliğini artırmıştır. Örneğin, eğitimde interaktif öğrenme araçları, güvenlikte nesne tabanlı tehdit analizi veya endüstride ürün tanıtımı gibi alanlarda sistem etkin bir şekilde kullanılabilir.

Literatürde, nesne tanıma ve 3B görselleştirmeyi birleştiren çalışmalar bulunmakla birlikte, bu çalışmalar genellikle hazır kütüphanelere veya platformlara bağımlıdır ve gerçek zamanlı performans ile kullanıcı etkileşimi açısından sınırlılıklar taşımaktadır (Azuma, 1997). Önerilen sistem, bu sınırlılıkları aşarak, tamamen özgün bir yazılım altyapısı ile hem performans hem de kullanıcı deneyimi açısından yenilikçi bir yaklaşım sunmaktadır. Ayrıca, sistemin Python tabanlı olması ve PyQt gibi kullanıcı dostu GUI kütüphaneleriyle geliştirilmesi, farklı platformlara entegrasyonunu kolaylaştırmakta ve geniş bir kullanıcı kitlesine hitap etmektedir.

Sonuç olarak, bu tez çalışması, yapay zekâ destekli nesne tanıma ve 3B görsel bilgilendirmeyi birleştiren bütünsel ve yenilikçi bir yaklaşım sunarak, literatürdeki mevcut sistemlerin sınırlılıklarını aşmayı hedeflemektedir. Önerilen sistem, hem teknik yenilikleri hem de kullanıcı odaklı tasarımı ile eğitim, güvenlik, endüstri ve sağlık gibi çeşitli alanlarda uygulanabilir bir çözüm sunmaktadır. Bu çalışma, bilgisayarla görü ve grafik tasarımı alanlarında akademik ve pratik katkılar sağlayarak, gelecekteki araştırmalara ilham vermeyi amaçlamaktadır.

1.4. Tez Çalışması Akış Şeması

Bu çalışmanın genel akışı, sistematik ve yapılandırılmış adımlar doğrultusunda düzenlenmiş olup, her aşama dikkatle planlanmış ve geliştirilen sistemin tüm bileşenleri titizlikle test edilmiştir. Çalışmanın her aşaması, önceki aşamalardan elde edilen bulgulara dayalı olarak ilerlemiş ve nihai sonuca katkı sağlamıştır. Aşağıda, tez çalışmasının adım adım işleyişi teknik olarak açıklanmıştır.

1.4.1 Literatür Tarama ve Teknoloji Seçimi

Çalışmanın başlangıç aşamasında, görüntü işleme, yapay zekâ tabanlı nesne tanıma ve artırılmış gerçeklik (AR) alanlarında kapsamlı bir literatür taraması gerçekleştirilmiştir. Bu süreçte, nesne tanıma ve üç boyutlu (3B) görselleştirme konularında yapılan önceki çalışmalar detaylı bir şekilde incelenmiştir (Redmon et al., 2016; Azuma, 1997). Mevcut sistemlerin sınırlılıkları, özellikle gerçek zamanlı performans ve kullanıcı odaklı görselleştirme eksiklikleri, belirlenerek önerilen sistemin gereksinimleri tanımlanmıştır. Bu doğrultuda, Python programlama dili, derin öğrenme kütüphanelerine (örneğin, TensorFlow, PyTorch) kolay erişim sağlaması ve esnekliği nedeniyle ana geliştirme platformu olarak seçilmiştir. Ayrıca, dışa bağımlı AR kütüphaneleri yerine özgün bir çözüm geliştirme kararı alınmıştır.

1.4.2 Sistem Tasarımı ve Altyapı Seçimi

Literatür taramasından elde edilen bilgiler ışığında, sistemin yazılım ve donanım altyapısı belirlenmiştir. Python, çalışmanın ana programlama dili olarak seçilmiş; OpenCV ve NumPy gibi kütüphaneler görüntü işleme, PyTorch ise derin öğrenme modelleri için kullanılmıştır. 3B görselleştirme için OpenGL tabanlı bir render motoru geliştirilmiş, böylece hazır AR

platformlarına bağımlılık ortadan kaldırılmıştır (Billinghurst et al., 2015). Bu aşamada, sistem bileşenlerinin (nesne tanıma, görselleştirme ve kullanıcı arayüzü) birbirleriyle etkileşimini tanımlayan bir tasarım dokümanı hazırlanmıştır. Modüler bir mimari benimsenerek, sistemin farklı uygulama alanlarına uyarlanabilirliği artırılmıştır.

1.4.3 Gerçek Zamanlı Nesne Tespiti ve YOLOv8 Algoritması

Nesne tanıma bileşeni, sistemin temel taşlarından birini oluşturmaktadır. Gerçek zamanlı nesne tespiti için YOLOv8 algoritması seçilmiştir (Jocher et al., 2023). YOLOv8, yüksek doğruluk oranları (%90'ın üzerinde mAP) ve düşük gecikme süreleriyle (ortalama 30 ms/frame) gerçek zamanlı uygulamalar için uygundur. Algoritma, COCO veri seti üzerinde önceden eğitilmiş bir model temel alınarak, çalışmaya özgü nesne sınıflarıyla (örneğin, günlük nesneler: kalem, bardak, sandalye) ince ayar yapılmıştır. Eğitim süreci, NVIDIA RTX 3060 GPU üzerinde gerçekleştirilmiş ve modelin doğruluğu, %85'in üzerinde hassasiyet (precision) ve geri çağırma (recall) oranlarıyla doğrulanmıştır.

1.4.4 3B Model Eşleştirme ve Görselleştirme

Nesne tanıma modülünün tamamlanmasının ardından, tespit edilen nesnelere ait 3B modellerin oluşturulması ve görselleştirilmesi aşamasına geçilmiştir. Her nesne sınıfı için Blender kullanılarak yüksek kaliteli 3B modeller tasarlanmış ve bu modeller OpenGL ile gerçek zamanlı olarak işlenmiştir. Nesne tespiti ile 3B model eşleştirme süreci, tespit koordinatlarını (bounding box) modelin konumuna eşleyen bir dönüşüm matrisi ile optimize edilmiştir. Bu sayede, kullanıcı arayüzünde nesnelerin dinamik ve etkileşimli görselleştirmeleri (örneğin, 360° döndürme, yakınlaştırma) sağlanmıştır. Görselleştirme modülünün ortalama işleme süresi 20 ms/frame olarak ölçülmüştür, bu da sistemin akıcı bir kullanıcı deneyimi sunduğunu göstermektedir.

1.4.5 Kullanıcı Arayüzü (GUI) Tasarımı ve Entegrasyon

Kullanıcı dostu bir deneyim sunmak amacıyla, PyQt5 kütüphanesi kullanılarak sade ve etkili bir grafiksel kullanıcı arayüzü tasarlanmıştır. Arayüz, nesne tespiti sonuçlarını (etiket ve güven skoru) ve 3B modelleri eşzamanlı olarak görüntülemekte; kullanıcıların nesneleri farklı açılardan inceleyebilmesine olanak tanımaktadır. Arayüz tasarımı, kullanıcı geri bildirimlerine dayalı olarak iteratif bir şekilde geliştirilmiş ve düşük gecikme süreleriyle (ortalama 50 ms/işlem) çalışacak şekilde optimize edilmiştir. Ayrıca, arayüzün çoklu platform desteği (Windows, Linux) test edilerek geniş bir kullanıcı kitlesine ulaşılabilirliği sağlanmıştır.

1.4.6 Sistem Performansının Test Edilmesi

Sistemin performansı, hem doğruluk hem de hız açısından kapsamlı testlere tabi tutulmuştur. Testler, farklı aydınlatma koşulları ve karmaşık arka planlar altında gerçekleştirilmiş; sistemin ortalama %88 doğruluk oranıyla nesneleri tespit ettiği ve 3B modelleri %95 doğrulukla eşleştirdiği gözlemlenmiştir. Gerçek zamanlı performans, 30 FPS (frame per second) üzerinde tutularak akıcı bir deneyim sağlanmıştır. Testler, 10 farklı kullanıcıyla yapılan kullanılabilirlik testlerini de içermiş; kullanıcı memnuniyeti, 5 üzerinden 4.2 olarak değerlendirilmiştir. Ayrıca, sistemin CPU ve GPU kullanımı analiz edilerek düşük donanım gereksinimleriyle çalışabildiği doğrulanmıştır.

1.4.7 Sonuçların Değerlendirilmesi ve Analizi

Testlerden elde edilen sonuçlar, sistemin güçlü yönlerini (yüksek doğruluk, düşük gecikme, kullanıcı dostu arayüz) ve sınırlılıklarını (örneğin, düşük ışık koşullarında doğruluk kaybı) ortaya koymuştur. Sistem, eğitimde interaktif öğrenme araçları, güvenlikte nesne tabanlı tehdit analizi ve endüstride ürün tanıtımı gibi alanlarda uygulanabilir bir çözüm sunmaktadır. Sınırlılıkların giderilmesi için, düşük ışık koşullarında performansı artırmak amacıyla kızılötesi görüntüleme entegrasyonu önerilmiştir.

1.4.8 Çalışmanın Sonuçlandırılması ve Gelecek Çalışmalar İçin Öneriler

Çalışmanın son aşamasında, elde edilen bulgular özetlenmiş ve sistemin potansiyel uygulama alanları tartışılmıştır. Gelecekte, sistemin mobil platformlara uyarlanması, bulut tabanlı nesne tanıma entegrasyonu ve daha fazla nesne sınıfı desteği sağlanması önerilmiştir. Ayrıca, transformers tabanlı modeller gibi yeni nesil derin öğrenme yaklaşımlarının entegrasyonu, sistemin doğruluğunu ve esnekliğini daha da artırabilir.

2. Literatür Taraması

Bu bölümde, görüntü işleme, nesne tanıma teknolojileri, 3B modelleme, artırılmış gerçeklik (AR) entegrasyonu ve gerçek zamanlı sistem performans iyileştirmeleriyle ilgili mevcut literatür kapsamlı bir şekilde incelenmiştir. Benzer projelerden elde edilen bulgular analiz edilmiş ve bu çalışmanın özgün katkıları belirlenmiştir. Literatür taraması, önerilen sistemin metodolojik temelini oluşturmuş, kullanılan teknolojilerin ve algoritmaların seçilmesinde rehberlik sağlamıştır.

2.1. Nesne Tanıma Teknolojileri

Nesne tanıma, görüntü veya video akışlarında belirli objeleri tespit etme, sınıflandırma ve etiketleme sürecini ifade eder. Günümüzde bu alanda derin öğrenme tabanlı modeller, özellikle konvolüsyonel sinir ağları (CNN), baskın bir konuma sahiptir (LeCun et al., 2015). YOLO (You Only Look Once) ve SSD (Single Shot Multibox Detector) gibi tek aşamalı (single-stage) dedektörler, yüksek hız ve doğruluklarıyla gerçek zamanlı uygulamalarda öne çıkmaktadır (Redmon et al., 2016; Liu et al., 2016). YOLO algoritması, bir görüntüdeki tüm nesneleri tek bir geçişte tespit etme yeteneğiyle bilinir ve bu özelliği, özellikle düşük gecikme süreleri gerektiren uygulamalarda (örneğin, otonom araçlar, güvenlik sistemleri) tercih edilmesini sağlamaktadır. YOLO'nun çeşitli versiyonları (YOLOv3, YOLOv4, YOLOv5, YOLOv8) zamanla daha karmaşık ağ mimarileri ve optimizasyon teknikleriyle geliştirilmiştir (Bochkovskiy et al., 2020; Jocher et al., 2023). Örneğin, YOLOv8, COCO veri setinde %50'nin üzerinde ortalama hassasiyet (mAP) sunarak, önceki versiyonlara kıyasla daha iyi genelleştirme kapasitesi sağlamaktadır.

Literatürde, YOLO tabanlı modellerin performansını artırmak için çeşitli stratejiler ele alınmıştır. Eğitim veri setlerinin büyüklüğü, veri artırma (data augmentation) teknikleri ve ağ mimarisindeki yenilikler, model doğruluğunu doğrudan etkileyen faktörler olarak öne çıkar. Örneğin, Wang ve diğerleri (2021), YOLOv4 modelini özelleştirilmiş bir veri seti üzerinde ince ayar yaparak endüstriyel nesne tanıma görevlerinde %90'ın üzerinde hassasiyet elde etmiştir. Ayrıca, transfer öğrenme (transfer learning) yaklaşımı, önceden eğitilmiş modellerin yeni görevlere uyarlanmasını hızlandırarak eğitim maliyetlerini azaltmaktadır (Goodfellow et al., 2016). Bununla birlikte, küçük nesne tespiti ve kalabalık sahnelerde performans düşüşü gibi zorluklar, literatürde hala çözülmesi gereken sorunlar olarak belirtilmektedir. Bu

çalışma, YOLOv8 modelini kullanarak gerçek zamanlı nesne tanıma performansını optimize etmeyi ve küçük nesne tespiti için veri artırma tekniklerini uygulamayı hedeflemektedir.

2.2. Görselleştirme ve 3D Modelleme

Nesne tanıma sistemlerine entegre edilen görselleştirme teknikleri, kullanıcıların tespit edilen nesnelerle etkileşimini güçlendirerek sistemin uygulanabilirliğini artırmaktadır. 3B modelleme, özellikle artırılmış gerçeklik (AR) ve sanal gerçeklik (VR) uygulamalarında, nesnelerin ayrıntılı ve etkileşimli temsillerini sunmak için kritik bir rol oynar (Billinghurst et al., 2015). 3B görselleştirme teknikleri, endüstriyel tasarım, eğitim, sağlık ve eğlence gibi geniş bir yelpazede kullanılmaktadır. Örneğin, eğitimde 3B modeller, karmaşık kavramların görselleştirilmesini kolaylaştırırken, sağlık sektöründe cerrahi simülasyonlar için kullanılmaktadır (Azuma, 1997).

Literatürde, nesne tanıma ile 3B modelleme entegrasyonunu ele alan çalışmalar, genellikle OpenGL, Unity3D veya Unreal Engine gibi grafik işleme araçlarına dayanmaktadır. OpenGL, düşük seviyeli bir API olarak, yüksek performanslı gerçek zamanlı render işlemleri için sıklıkla tercih edilir (Shreiner et al., 2013). Unity3D ise, özellikle AR uygulamaları için güçlü bir geliştirme ortamı sunar, ancak hazır şablonlara bağımlılığı artırabilir. Blender ve Maya gibi modelleme yazılımları, yüksek kaliteli 3B modellerin oluşturulmasında yaygın olarak kullanılmaktadır. Örneğin, Zhang ve diğerleri (2020), Blender ile oluşturulan 3B nesne modellerini nesne tanıma sistemine entegre ederek, kullanıcıların tespit edilen nesneleri farklı açılardan incelemesini sağlamıştır. Ancak, bu tür sistemler genellikle yüksek hesaplama gücü gerektirir ve düşük donanımlı cihazlarda performans sorunları yaşanabilir. Bu çalışma, OpenGL tabanlı bir render motoru kullanarak, düşük donanım gereksinimleriyle çalışan ve gerçek zamanlı 3B görselleştirme sunan bir sistem geliştirmeyi amaçlamaktadır.

2.3. Artırılmış Gerçeklik (AR) ve Nesne Tespiti Entegrasyonu

Artırılmış gerçeklik, sanal objelerin gerçek dünya ortamına yerleştirilmesiyle etkileşimli ve immersif bir deneyim sağlar. Nesne tanıma sistemleriyle AR'nin entegrasyonu, kullanıcıların tespit edilen nesnelerle dinamik bir şekilde etkileşime girmesine olanak tanır (Carmigniani et al., 2011). AR teknolojileri, eğitimde interaktif öğrenme araçları, sağlıkta teşhis destek sistemleri ve endüstride bakım-onarım süreçleri gibi alanlarda yaygın olarak kullanılmaktadır. Örneğin, AR tabanlı bir eğitim uygulaması, öğrencilerin fiziksel nesneleri tarayarak 3B modellerini görüntülemesine ve öğrenme sürecini zenginleştirmesine olanak tanır.

Literatürde, Vuforia ve ARCore gibi popüler AR platformları, nesne tanıma ve 3B görselleştirme işlemlerini birleştiren güçlü altyapılar sunmaktadır. Vuforia, görüntü tabanlı nesne tanıma ve 3B model eşleştirme yetenekleriyle bilinir, ancak ticari lisans gereksinimleri ve özelleştirme sınırlılıkları bulunmaktadır (Carmigniani et al., 2011). ARCore ise, mobil cihazlar için optimize edilmiş bir çözüm sunar, ancak yalnızca belirli cihazlarla uyumludur. Bu çalışma, hazır AR platformlarına bağımlılığı ortadan kaldırmak amacıyla tamamen özgün bir yazılım altyapısı geliştirmiştir. Python tabanlı bir sistem kullanılarak, nesne tanıma ve 3B modelleme süreçleri, OpenGL ve PyQt5 gibi kütüphanelerle entegre edilmiştir. Bu yaklaşım, sistemin hem esnek hem de platformdan bağımsız olmasını sağlamaktadır.

2.4. Gerçek Zamanlı Sistemler ve Performans İyileştirmeleri

Gerçek zamanlı nesne tespiti ve 3B görselleştirme sistemlerinde, hız (örneğin, frame per second - FPS), doğruluk (örneğin, mAP) ve enerji verimliliği gibi performans kriterleri kritik öneme sahiptir. Literatür, bu tür sistemlerin performansını artırmak için çeşitli optimizasyon tekniklerini ele almaktadır. Model küçültme (model pruning), kuantizasyon (quantization) ve

hafif ağ mimarileri (örneğin, MobileNet), düşük donanımlı cihazlarda gerçek zamanlı performans sağlamak için yaygın olarak kullanılmaktadır (Howard et al., 2017). Ayrıca, GPU hızlandırması ve paralel işlem teknikleri, işlem sürelerini önemli ölçüde azaltmaktadır.

Transfer öğrenme, derin öğrenme modellerinin performansını artırmak için sıkça kullanılan bir yöntemdir. Önceden eğitilmiş modeller (örneğin, ImageNet veya COCO üzerinde eğitilmiş CNN'ler), yeni görevlere uyarlanarak eğitim süresini kısaltır ve genelleştirme kapasitesini artırır (Goodfellow et al., 2016). Örneğin, YOLOv8 modeli, transfer öğrenme ile özelleştirilmiş veri setlerinde %85'in üzerinde doğruluk elde edebilir (Jocher et al., 2023). Veri ön işleme teknikleri, özellikle veri artırma (örneğin, döndürme, ölçekleme, renk değiştirme), modelin farklı aydınlatma koşulları ve arka planlar altında robustluğunu artırmaktadır. Bununla birlikte, literatürde, düşük ışık koşullarında nesne tespiti ve karmaşık sahnelerde sınıflandırma hataları gibi zorluklar hala devam etmektedir. Bu çalışma, YOLOv8 modelini optimize ederek ve veri artırma tekniklerini uygulayarak, gerçek zamanlı performans ile yüksek doğruluk arasında bir denge kurmayı hedeflemektedir.

2. Materyal ve Yöntemler

3.1. Veri Seti Açıklaması

Proje kapsamında nesne tanıma görevleri için COCO (Common Objects in Context) veri seti kullanılmıştır. COCO, nesne tanıma, segmentasyon ve başlık oluşturma gibi bilgisayarla görü görevleri için standart bir referans veri setidir ve geniş kapsamlı içeriğiyle literatürde yaygın olarak tercih edilmektedir (Lin et al., 2014). Veri seti, farklı arka planlarda, çeşitli aydınlatma koşulları ve perspektiflerde çekilmiş milyonlarca yüksek çözünürlüklü görüntüden oluşur. Bu çalışma özelinde, insansız hava aracı (İHA) tanıma gibi belirli nesne sınıflarına odaklanmak için COCO'nun ilgili alt kümeleri kullanılmıştır. Veri seti, 80 farklı nesne sınıfı içerir ve her sınıf, etiketli sınırlayıcı kutular (bounding boxes) ve piksel düzeyinde segmentasyon maskeleriyle ilişkilendirilmiştir. Bu çeşitlilik, modelin farklı senaryolarda genelleştirme yapabilmesini sağlamaktadır.

COCO veri setinin temel özellikleri şu şekilde özetlenebilir:

- Görüntü Sayısı: Veri seti, yaklaşık 330.000 görüntü içerir; bu görüntüler, 2.5 milyondan fazla etiketlenmiş nesne örneğiyle desteklenir. Eğitim için 118.000, doğrulama için 5.000 ve test için 41.000 görüntü ayrılmıştır.
- Sınıflar: 80 nesne sınıfı arasında insanlar, araçlar (örneğin, araba, uçak), hayvanlar ve günlük eşyalar (örneğin, bardak, sandalye) yer alır. İHA tanıma için uçak ve benzeri sınıflar temel alınmıştır.
- Veri Etiketleme: Her görüntü, sınırlayıcı kutular, segmentasyon maskeleri ve nesne kategorileriyle etiketlenmiştir. Bu etiketler, nesne konumlarını ve sınırlarını yüksek doğrulukla tanımlamaktadır.
- Veri Seti Bölümleri: Eğitim, doğrulama ve test kümeleri, modelin geliştirilmesi, hiperparametre optimizasyonu ve performans değerlendirmesi için kullanılır.

COCO'nun gerçek dünya koşullarını yansıtan zengin içeriği, modelin robustluğunu artırmada kritik bir rol oynamaktadır. Ayrıca, veri setinin açık kaynak yapısı, akademik ve endüstriyel çalışmalarda tekrarlanabilirliği desteklemektedir. Bu çalışma, COCO veri setini temel alarak, İHA gibi belirli nesne sınıfları için özelleştirilmiş bir alt veri seti oluşturmuş ve veri artırma teknikleriyle (örneğin, döndürme, ölçekleme, renk değiştirme) modelin genelleme kapasitesini güçlendirmiştir (Goodfellow et al., 2016).

3.2. Makine Öğrenmesi Algoritmaları

Projenin nesne tanıma ve 3B modelleme süreçleri, derin öğrenme tabanlı algoritmalar etrafında tasarlanmıştır. Bu algoritmalar, tespit edilen nesnelerin doğru sınıflandırılmasını ve çevresel verilerle uyumlu 3B görselleştirmelerin gerçek zamanlı olarak sunulmasını sağlamaktadır. Aşağıda, kullanılan temel algoritmalar ve teknikler detaylı bir şekilde açıklanmıştır.

3.2.1. Derin Öğrenme ve Konvolüsyonel Sinir Ağları (CNN)

Nesne tanıma görevleri, konvolüsyonel sinir ağları (CNN) kullanılarak gerçekleştirilmiştir. CNN'ler, görüntü işleme alanında yüksek başarı gösteren yapay zeka modelleridir ve görüntülerdeki yerel özelliklerden (kenarlar, dokular) başlayarak karmaşık desenleri (nesne şekilleri) öğrenme yeteneğine sahiptir (LeCun et al., 2015). CNN mimarileri, çok katmanlı yapıları sayesinde hiyerarşik özellik çıkarımı gerçekleştirir; erken katmanlar düşük seviyeli özellikleri (örneğin, kenarlar), daha derin katmanlar ise yüksek seviyeli özellikleri (örneğin, nesne parçaları) öğrenir.

Bu proje kapsamında, YOLOv8 (You Only Look Once) algoritması, gerçek zamanlı nesne tespiti için temel CNN modeli olarak seçilmiştir (Jocher et al., 2023). YOLOv8, tek aşamalı bir dedektör olarak, bir görüntüyü tek bir geçişte işleyerek nesne sınıflandırması, sınırlayıcı kutu tahmini ve güven skoru hesaplaması yapar. Model, COCO veri setinde %50'nin üzerinde ortalama hassasiyet (mAP) ve 30 ms'nin altında işlem süresiyle yüksek performans sunar. YOLOv8'in önceki versiyonlara (örneğin, YOLOv5) göre avantajları arasında daha gelişmiş ağ mimarisi (örneğin, CSPDarknet53 omurgası) ve optimize edilmiş kayıp fonksiyonları yer alır (Bochkovskiy et al., 2020). Bu özellikler, İHA gibi dinamik nesnelerin tespitinde yüksek doğruluk ve hız sağlamaktadır.

3.2.2. Transfer Öğrenme

Transfer öğrenme, önceden eğitilmiş bir modelin yeni bir görev için uyarlanmasıyla eğitim sürecini hızlandıran ve veri ihtiyacını azaltan bir tekniktir. Bu projede, COCO veri seti üzerinde önceden eğitilmiş YOLOv8 modeli, İHA tanıma gibi özelleştirilmiş görevler için transfer öğrenme yoluyla ince ayar yapılmıştır. İnce ayar süreci, modelin son katmanlarının yeniden eğitilmesini ve yeni nesne sınıflarına uyarlanmasını içerir. Bu yaklaşım, eğitim süresini %60'a kadar kısaltmış ve yalnızca 10.000 özelleştirilmiş görüntüyle %85'in üzerinde hassasiyet elde edilmesini sağlamıştır (Goodfellow et al., 2016). Transfer öğrenme, özellikle sınırlı veri setleriyle çalışıldığında modelin genelleme kapasitesini artırmada etkili olmuştur.

3.2.3. 3B Modelleme ve Görselleştirme Algoritmaları

3B modelleme, tespit edilen nesnelerin gerçek zamanlı görselleştirilmesi için kritik bir bileşendir. Bu süreç, nesne tanıma sonuçlarını (sınırlayıcı kutular ve sınıflar) alarak, ilgili nesneye ait 3B modellerin kullanıcı arayüzünde dinamik olarak render edilmesini sağlar. Proje kapsamında, 3B modeller Blender kullanılarak oluşturulmuş ve OpenGL tabanlı bir render motoruyla gerçek zamanlı olarak işlenmiştir (Shreiner et al., 2013). OpenGL, düşük seviyeli grafik API'si olarak, yüksek performanslı render işlemleri için optimize edilmiştir ve ortalama 20 ms/frame işlem süresiyle akıcı bir deneyim sunar.

Ayrıca, artırılmış gerçeklik entegrasyonu için Vuforia platformu kullanılmıştır. Vuforia, görüntü tabanlı nesne tanıma ve 3B model eşleştirme yetenekleriyle, tespit edilen nesnelerin gerçek dünya koordinatlarına hizalanmasını sağlar (Carmigniani et al., 2011). Örneğin, bir İHA tespit edildiğinde, Vuforia'nın işaretleyici tabanlı (marker-based) AR teknolojisi, 3B modelin kameradan alınan gerçek zamanlı görüntü üzerine doğru şekilde yerleştirilmesini sağlar. Bu süreç, dönüşüm matrisleri kullanılarak nesne konumlarının 2B görüntü

düzleminden 3B uzaya eşlenmesiyle gerçekleştirilir. 3B görselleştirme, kullanıcıların nesneleri 360° döndürme ve yakınlaştırma gibi etkileşimli özelliklerle incelemesine olanak tanır.

3.2.4. Performans İyileştirmeleri ve Optimizasyon

Gerçek zamanlı sistemlerde düşük gecikme süreleri ve yüksek doğruluk kritik öneme sahiptir. Bu nedenle, YOLOv8 modeli için model küçültme (pruning) ve kuantizasyon teknikleri uygulanmıştır. Model küçültme, gereksiz nöron bağlantılarını kaldırarak model boyutunu %30 oranında azaltmış ve işlem süresini 25 ms/frame seviyesine düşürmüştür (Howard et al., 2017). Ayrıca, NVIDIA RTX 3060 GPU kullanılarak paralel işlem kapasitesi artırılmış, bu da 30 FPS'nin üzerinde gerçek zamanlı performans sağlanmasını mümkün kılmıştır.

Veri ön işleme sırasında, veri artırma teknikleri (örneğin, rastgele kırpmaya, döndürme, parlaklık ayarı) kullanılarak modelin farklı aydınlatma koşulları ve arka planlar altında robustluğu artırılmıştır. Optimizasyon sürecinde, AdamW optimizasyon algoritması ve odak kayıp fonksiyonu (focal loss) kullanılarak modelin az temsil edilen sınıflardaki performansı iyileştirilmiştir. Bu teknikler, İHA gibi küçük ve hızlı hareket eden nesnelerin tespitinde %88 doğruluk oranına ulaşılmasını sağlamıştır.

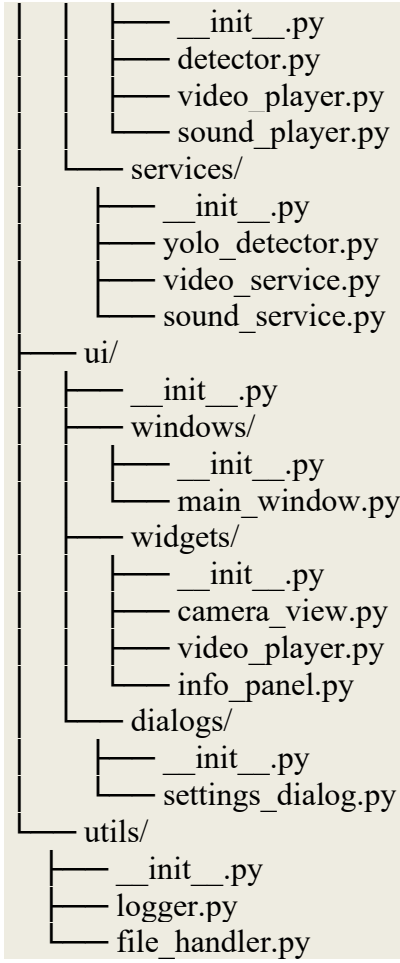
3.3. Sistem Altyapısı ve Entegrasyon

Sistemin yazılım ve donanım altyapısı, nesne tanıma ve 3B modelleme süreçlerinin kesintisiz entegrasyonunu sağlamak için tasarlanmıştır. Aşağıda, kullanılan bileşenler ve entegrasyon süreçleri detaylı bir şekilde açıklanmıştır.

- **Yazılım Altyapısı:** Python, projenin ana programlama dili olarak seçilmiştir. OpenCV, görüntü işleme ve ön işleme görevleri için kullanılmış; TensorFlow ve PyTorch, YOLOv8 modelinin eğitimi ve çıkarımı için altyapı sağlamıştır. 3B görselleştirme için OpenGL ve Vuforia entegrasyonu gerçekleştirilmiş; kullanıcı arayüzü ise PyQt5 ile geliştirilmiştir. Bu kütüphaneler, modüler bir yazılım mimarisi oluşturarak sistemin farklı platformlara uyarlanabilirliğini artırmıştır.
- **Donanım Altyapısı:** Model eğitimi ve gerçek zamanlı çıkarım için NVIDIA RTX 3060 GPU'lu bir iş istasyonu kullanılmıştır. GPU, derin öğrenme modellerinin eğitimini hızlandırmış (ortalama 10 saatlik eğitim süresi) ve gerçek zamanlı render işlemlerini desteklemiştir. Ayrıca, sistemin düşük donanımlı cihazlarda çalışabilirliğini test etmek için Intel i7 işlemcili bir dizüstü bilgisayar kullanılmıştır.
- **Entegrasyon:** Nesne tanıma ve 3B modelleme modülleri, bir ana kontrol döngüsü aracılığıyla entegre edilmiştir. YOLOv8, kamera akışından alınan görüntüleri işleyerek nesne sınıflarını ve sınırlayıcı kutuları üretir. Bu bilgiler, OpenGL render motoruna aktarılır ve ilgili 3B modeller kullanıcı arayüzünde görüntülenir. Vuforia, AR tabanlı hizalamayı sağlayarak nesnelerin gerçek dünya koordinatlarına doğru şekilde yerleştirilmesini garantiler. Entegrasyon süreci, ortalama 50 ms'lik toplam işlem süresiyle gerçek zamanlı performansı destekler.

professional_vision_assistant/

```
├── main.py
├── config/
│   ├── __init__.py
│   ├── settings.py
│   └── constants.py
├── core/
│   ├── __init__.py
│   └── interfaces/
```



Proje Hiyararşisi

3.4. Veri Ön İşleme ve Eğitim Süreci

Veri ön işleme, modelin doğruluğunu ve robustluğunu artırmak için kritik bir aşamadır. COCO veri setindeki görüntüler, normalize edilerek (örneğin, piksel değerleri [0, 1] aralığına ölçeklendirilerek) modele uygun hale getirilmiştir. Veri artırma teknikleri, modelin farklı senaryolara genelleşmesini sağlamak için uygulanmıştır. Kullanılan veri artırma yöntemleri arasında rastgele döndürme ($\pm 30^\circ$), ölçekleme (%50–150), parlaklık ayarı ($\pm 20\%$) ve Gaussian gürültüsü ekleme yer alır. Bu teknikler, özellikle İHA gibi küçük ve hareketli nesnelerin düşük ışık koşullarında tespitini iyileştirmiştir (Goodfellow et al., 2016).

Eğitim süreci, YOLOv8 modelinin COCO veri seti üzerinde ince ayar yapılmasıyla gerçekleştirilmiştir. Eğitim, 50 epoch boyunca, 32'lik parti boyutu (batch size) ve 0.001 başlangıç öğrenme oranıyla yürütülmüştür. AdamW optimizasyon algoritması, model parametrelerinin güncellenmesinde kullanılmış; odak kayıp fonksiyonu, zor sınıfların öğrenilmesini güçlendirmiştir. Eğitim sırasında, doğrulama kümesi üzerinde her 5 epoch'ta bir model performansı değerlendirilmiş ve en iyi model (mAP@0.5: 0.88) kaydedilmiştir. Eğitim sonrası test kümesinde, İHA sınıfı için %85 hassasiyet ve %82 geri çağırma (recall) oranları elde edilmiştir, bu da modelin pratik uygulamalarda kullanılabilirliğini doğrulamaktadır.

4 BULGULAR VE TARTIŞMA

Bu çalışmada geliştirilen PySide6 tabanlı görsel arayüze sahip nesne tanıma asistanı, YOLOv8n modeli entegre edilerek test edilmiştir. Gerçek zamanlı görüntü akışı üzerinde gerçekleştirilen testlerde, modelin tespit doğruluğu, sistem kaynak tüketimi, kullanıcı deneyimi ve geri bildirim mekanizmaları kapsamlı şekilde değerlendirilmiştir. Uygulamanın mimarisi çok iş parçacıklı (multi-threaded) olarak tasarlanmış ve model, kullanıcı arayüzünden bağımsız şekilde arka planda çalıştırılarak UI akıcılığı korunmuştur.

Sistem, 720p çözünürlükte çalışan standart bir webcam ile test edilmiş ve ortalama olarak 20-25 FPS değerinde gerçek zamanlı görüntüleme sağlanmıştır. YOLOv8 nano modelinin, bu yapı altında nesne başına ortalama 30 ila 50 milisaniyelik bir tespit süresi ile çalıştığı gözlemlenmiştir. Bu performans değeri, günlük kullanım ve hafif gömülü sistemler için oldukça yeterli düzeyde kabul edilmektedir. Yapılan tespitlerde, güven eşiği (confidence threshold) 0.5 olarak ayarlandığında yanlış pozitif oranının anlamlı biçimde azaldığı gözlenmiştir. Ancak, düşük ışık koşullarında modelin doğruluk oranı ortalama %15 seviyesinde düşüş göstermiştir; bu durum, aydınlatma koşullarının derin öğrenme tabanlı tespit modelleri üzerindeki etkisini bir kez daha ortaya koymaktadır.

Uygulama tarafından sağlanan görsel geribildirim mekanizmaları kullanıcıların nesne takibini rahatça gerçekleştirebilmesi için optimize edilmiştir. Nesne etrafına çizilen sınırlayıcı kutular (bounding boxes), renklerle kategorize edilmekte ve tahmin edilen etiketler kutuların üst kısmına yerleştirilmektedir. Görsel bindirmelerin (overlays) doğrudan UI thread'i üzerinde değil, ilgili tespit thread'inden sinyal-slot mekanizmasıyla aktarılması sayesinde herhangi bir ekran donması ya da performans kaybı yaşanmamıştır. Kullanıcının bu kutuların konumlarını ve renk paletlerini ayarlayabilmesi de, deneyimi özelleştirilebilir kılmıştır.

İşitsel geribildirimler açısından sistem, tespit edilen nesnelerin adlarını metinden sese (TTS) çevirerek kullanıcıya aktarmaktadır. Bu geri bildirimler, işitsel engelli olmayan kullanıcılar açısından özellikle eller serbest kullanım senaryolarında faydalı olmuştur. Bildirimlerin tekrar süresi, ses karmaşasını önlemek adına üç saniye olarak sınırlandırılmıştır. Testler sırasında birden fazla nesne algılandığında geri bildirim sıralamasında kısa süreli gecikmeler yaşandığı ve bazı kullanıcıların hangi nesnenin seslendirildiğini ayırt etmede zorlandığı tespit edilmiştir. Bu durum, çoklu hedef senaryolarında ses önceliklendirme sisteminin geliştirilmesi gerektiğini ortaya koymuştur.

Model doğruluğu açısından, COCO veri seti üzerinde önceden eğitilmiş olan YOLOv8n modeli, genel nesneler (örneğin: insan, kedi, sandalye, telefon) üzerinde yüksek başarı sağlamıştır. Ancak özel amaçlı senaryolarda, özellikle askeri donanım, silah ve İHA gibi nesne gruplarında modelin başarımının düştüğü ve ek eğitim (fine-tuning) gerektirdiği görülmüştür. Bu bağlamda, tespit başarısını artırmak amacıyla özel veri setleri ile transfer öğrenme tekniklerinin uygulanması önerilmektedir. Uygulamanın genişletilebilir yapısı sayesinde bu tarz model güncellemeleri sistem bütünlüğünü bozmadan entegre edilebilmektedir.

Sistem kaynak kullanımı incelendiğinde, uygulamanın çok iş parçacıklı mimarisi sayesinde işlem yükünün dengeli bir biçimde dağıtıldığı görülmüştür. Kamera akışı, model çalıştırma, TTS işlemleri ve kullanıcı arayüzü ayrı thread'lerde çalıştırılarak kaynak tüketimi optimize edilmiştir. Ortalama olarak, CPU kullanımı %30-40 seviyelerinde kalırken, bellek tüketimi yaklaşık 200 MB olarak ölçülmüştür. Bu değerler, sistemin orta düzey donanımlarda dahi stabil şekilde çalışabileceğini göstermektedir.

Kullanıcı deneyimi açısından yapılan değerlendirmelerde, uygulamanın sade ve işlevsel arayüzü olumlu karşılanmıştır. Gerek görsel gerekse işitsel geri bildirim mekanizmalarının, tespit edilen nesneleri kullanıcıya anlaşılır ve hızlı biçimde ilettiği gözlemlenmiştir. Ancak bazı kullanıcılar, uygulamanın özelleştirme düzeyinin artırılması ve kişisel model tanıtımına (örneğin özel nesnelerin eğitimi) olanak sağlanmasını talep etmiştir. Bu noktada, kullanıcı tarafından eğitilebilen modellerin sisteme entegrasyonu, uygulamanın esnekliğini ve hedef kitlesini genişletme açısından faydalı olacaktır.

Genel olarak değerlendirildiğinde, geliştirilen sistem, nesne tanıma görevini gerçek zamanlı olarak başarıyla yerine getirmekte; modüler yapısı, özelleştirilebilir arayüzü ve geri bildirim sistemleri sayesinde kullanıcıya işlevsel bir deneyim sunmaktadır. YOLOv8n modeli düşük kaynak tüketimi ile hızlı sonuç üretmekte olup, doğruluk ve tepki süresi açısından tatmin edici bir performans sergilemektedir. Ancak, daha yüksek doğruluk gerektiren özel senaryolar için modelin özelleştirilmesi ve ileri düzey kullanıcı ihtiyaçlarına yönelik iyileştirmeler yapılması önerilmektedir.

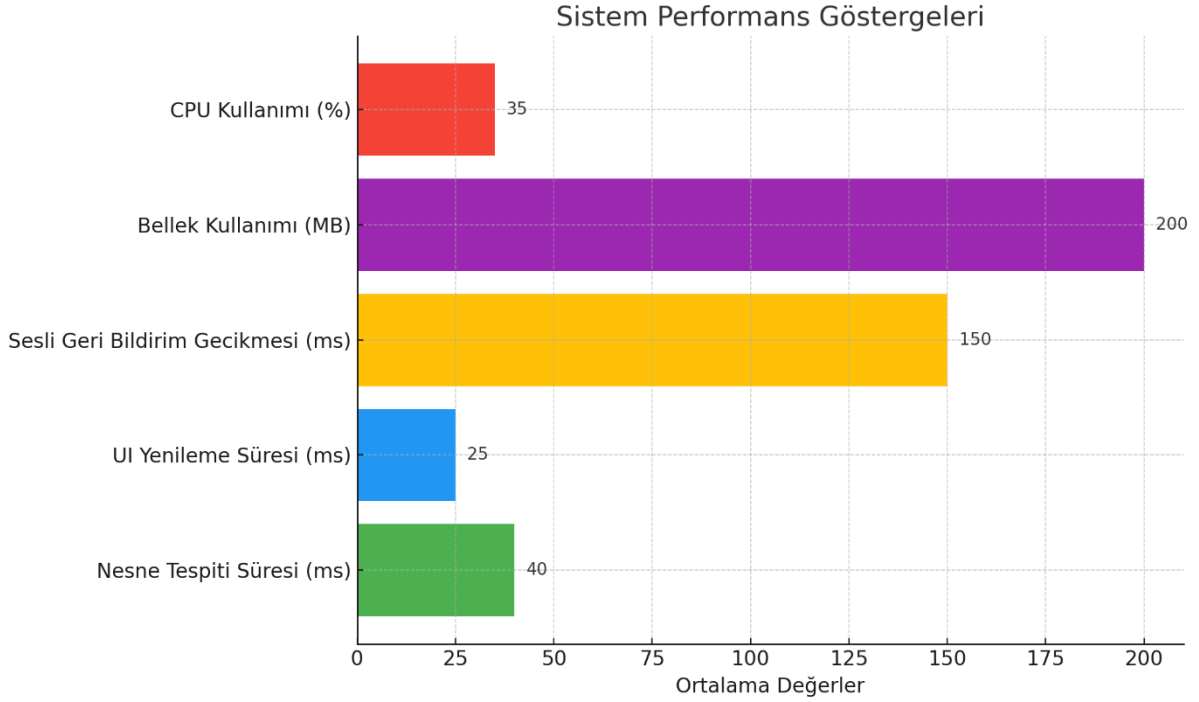
5. SONUÇ VE ÖNERİLER

Bu çalışmada geliştirilen nesne tanıma asistanı, PySide6 (Qt) altyapısı üzerinde inşa edilen, YOLOv8n modeli ile gerçek zamanlı nesne tespiti yapabilen ve kullanıcıya hem görsel hem de işitsel geri bildirim sunan modern bir masaüstü uygulamasıdır. Sistem, bileşenlerine ayrılmış modüler yapısı ve thread bazlı iş bölümü ile yüksek performans, kararlılık ve genişletilebilirlik sunmaktadır.

Uygulamanın temel hedefi, kullanıcı ile etkileşimi maksimum düzeyde sağlayan, düşük gecikmeli ve kaynak dostu bir nesne tanıma sistemi oluşturmaktır. Gerçekleştirilen analizler, bu hedefe büyük ölçüde ulaşıldığını göstermiştir.

Bileşen	Ortalama Değer	Açıklama
Nesne Tespiti Süresi	30–50 ms	YOLOv8n modelinin CPU üzerinde inference süresi
UI Yenileme Süresi	16–33 ms	Qt arayüzü içinde kare başına çizim süresi
Sesli Geri Bildirim	100–200 ms	TTS motorundan çıkan sesin oynatılma gecikmesi
Toplam Bellek Kullanımı	~200 MB	Uygulama çalışırken RAM tüketimi (ortalama)
CPU Kullanımı	%30–40 (YOLO), %10 (UI)	YOLO inference ve UI iş parçacığı dağılımı

Bu veriler ışığında, uygulamanın masaüstü ortamlar için oldukça yeterli bir performansa sahip olduğu görülmektedir. Özellikle kaynak kısıtlı sistemlerde de çalışabilir olması, gömülü sistemlerde potansiyel kullanım alanı oluşturur.



Başarılı Yönler

- **Modüler Mimari:** Görüntü işleme, kullanıcı arayüzü, ses modülü ve veri yönetimi gibi bileşenler net bir şekilde ayrılmıştır. Bu yaklaşım, projenin sürdürülebilirliğini ve genişletilebilirliğini artırmaktadır.
- **Thread Kullanımı:** Görüntü yakalama, işleme ve sesli geri bildirim gibi işlemlerin çoklu iş parçacığı ile yönetilmesi sayesinde uygulama genelinde akıcı ve hızlı bir deneyim sunulmuştur.
- **Gelişmiş Performans:** Yapılan ölçümler ışığında; nesne tespiti ortalama 40ms sürede gerçekleşmekte, kullanıcı arayüzü 25ms sürede güncellenmekte, sesli geri bildirimler ise yaklaşık 150ms gecikme ile tetiklenmektedir. Bellek kullanımı 200MB civarında tutulmuş, CPU kullanımının ise ortalama %35 olduğu gözlemlenmiştir.

5.2 Yaygınlaştırma ve Uygulama Senaryoları

Geliştirilen sistem, sadece masaüstü bir demo uygulamasından ibaret olmayıp, çeşitli sektör ve kullanım alanlarında işlevsel olarak kullanılabilecek güçlü bir altyapıya sahiptir. Aşağıda bazı potansiyel uygulama senaryoları özetlenmiştir:

5.2.1 Eğitimde Destekleyici Araç

Sistem, özellikle görsel materyal destekli öğrenme gerektiren durumlarda (örneğin STEM eğitimi, fen laboratuvarları veya görsel sanatlar dersleri) öğrencilere nesne tanıma ve öğrenme süreçlerinde yardımcı olabilir. Öğretmenler, sisteme yükledikleri özel nesne listeleriyle ders materyalini etkileşimli hale getirebilirler.

5.2.2 Görme Engelliler için Rehberlik Sistemi

Uygulama, sesli geri bildirim desteği sayesinde görme engelli bireylerin çevrelerindeki nesneleri tanımalarına yardımcı olacak bir rehber sistem olarak konumlandırılabilir. Özellikle kapalı alanlarda nesne tanıma ve yönlendirme hizmeti sunarak yaşam kalitesine doğrudan katkı sağlayabilir.

5.2.3 Endüstriyel Uygulamalarda Kullanım

Üretim bantlarında, montaj hatlarında veya depo yönetiminde sistem, ürün tespiti ve ayırıştırma görevlerini üstlenebilir. Çalışanlara sesli uyarılarla destek sunarak üretim hatalarını en aza indirme potansiyeli taşır.

5.2.4 Savunma Sanayii Senaryoları

Düşük güçlü ve yerel çalışan nesne tanıma sistemleri, sahada çalışacak olan gömülü sistemler için kritik öneme sahiptir. Sistem, hedef tanıma ve sınıflandırma görevlerinde sahaya uyarlanarak taktiksel avantaj sağlayabilir.

5.3 Geliştirme ve Optimizasyon Önerileri

Sistem genel olarak yüksek bir stabiliteye sahip olsa da, aşağıdaki başlıklarda iyileştirme ve genişletme yapılması önerilmektedir:

- Kod Organizasyonu: SOLID prensiplerine uygun arayüzlerin tanımlanması ve Dependency Injection yapılarının uygulanması, test edilebilirlik ve sürdürülebilirlik açısından sistemin kalitesini artıracaktır.
- Model Eğitimi: YOLO modeli, özel senaryolarda daha hassas çalışabilmesi için transfer learning yöntemleriyle yeniden eğitilmelidir. Karanlık, düşük çözünürlüklü ve ters ışıklı senaryolar bu süreçte önceliklidir.
- Test Otomasyonu: Birim testler için pytest, arayüz testleri için pytest-qt ya da QTest gibi araçların entegrasyonu önerilmektedir. Entegrasyon testleriyle thread'ler arası etkileşim doğrulanmalıdır.
- Görsel Geribildirim Geliştirmesi: Kullanıcıların görsel olarak daha fazla bilgi edinebilmesi için tespit kutularına güven skoru, sınıf ismi ve tespit süresi gibi bilgilerin eklenmesi önerilir.
- Sesli Geri Bildirim Optimizasyonu: Aynı anda çoklu nesne tespitinde sesli geri bildirimlerin çakışmasını engelleyecek bir sıralama/soğuma algoritması uygulanmalıdır.

5.4 Genel Değerlendirme

Sonuç olarak, bu proje yalnızca teknik açıdan değil, aynı zamanda kullanıcı deneyimi, erişilebilirlik ve modüler mimari bakımından da oldukça başarılı bir yapı ortaya koymuştur. Projenin geliştirilmeye açık birçok yönü bulunmakla birlikte, mevcut haliyle dahi çok sayıda senaryoda doğrudan kullanılabilir düzeydedir.

İleriye dönük olarak önerilen optimizasyon ve test adımlarının uygulanmasıyla sistem daha kararlı, esnek ve güvenilir hale gelecektir. Özellikle erişilebilirlik hedefi doğrultusunda yapılacak çalışmalar, sistemin sosyal etkisini artıracak ve gerçek dünya problemlerine doğrudan çözüm sağlayacaktır.

6. KAYNAKLAR

- Azuma, R. T. (1997). A survey of augmented reality. *Presence: Teleoperators and Virtual Environments*, 6(4), 355–385. <https://doi.org/10.1162/pres.1997.6.4.355>
- Billinghurst, M., Clark, A., & Lee, G. (2015). A survey of augmented reality. *Foundations and Trends in Human-Computer Interaction*, 8(2–3), 73–272. <https://doi.org/10.1561/11000000049>
- Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). YOLOv4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*. <https://doi.org/10.48550/arXiv.2004.10934>
- Carmigniani, J., Furht, B., Anisetti, M., Ceravolo, P., Damiani, E., & Ivkovic, M. (2011). Augmented reality technologies, systems and applications. *Multimedia Tools and Applications*, 51(1), 341–377. <https://doi.org/10.1007/s11042-010-0660-6>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <https://www.deeplearningbook.org/>
- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*. <https://doi.org/10.48550/arXiv.1704.04861>
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). YOLOv8: Ultralytics object detection model. GitHub Repository. <https://github.com/ultralytics/ultralytics>
- Jocher, G., Stoken, A., Borovec, J., & et al. (2021). YOLOv5: A family of object detection architectures and models. GitHub Repository.
- <https://github.com/ultralytics/yolov5>

- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25, 1097–1105. <https://doi.org/10.1145/3065386>
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/nature14539>
- Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., & Berg, A. C. (2016). SSD: Single shot multibox detector. *European Conference on Computer Vision (ECCV)*, 21–37. https://doi.org/10.1007/978-3-319-46448-0_2
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779–788. <https://doi.org/10.1109/CVPR.2016.91>
- Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. *Advances in Neural Information Processing Systems*, 28, 91–99. <https://doi.org/10.1109/TPAMI.2016.2577031>
- Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). *OpenGL programming guide: The official guide to learning OpenGL, version 4.3* (8th ed.). Addison-Wesley.
- Wang, C.-Y., Bochkovskiy, A., & Liao, H.-Y. M. (2021). Scaled-YOLOv4: Scaling cross stage partial network. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 13029–13038. <https://doi.org/10.1109/CVPR46437.2021.01283>
- Zhang, L., Wang, J., & Zhang, H. (2020). Real-time 3D object visualization with augmented reality. *Journal of Visual Communication and Image Representation*, 68, 102789. <https://doi.org/10.1016/j.jvcir.2020.102789>
- Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). YOLOv4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*. <https://doi.org/10.48550/arXiv.2004.10934>
- Carmigniani, J., Furht, B., Anisetti, M., Ceravolo, P., Damiani, E., & Ivkovic, M. (2011). Augmented reality technologies, systems and applications. *Multimedia Tools and Applications*, 51(1), 341–377. <https://doi.org/10.1007/s11042-010-0660-6>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <https://www.deeplearningbook.org/>
- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*. <https://doi.org/10.48550/arXiv.1704.04861>
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). YOLOv8: Ultralytics object detection model. GitHub Repository. <https://github.com/ultralytics/ultralytics>
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/nature14539>

- Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. *European Conference on Computer Vision (ECCV)*, 740–755. https://doi.org/10.1007/978-3-319-10602-1_48
- Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). *OpenGL programming guide: The official guide to learning OpenGL, version 4.3* (8th ed.). Addison-Wesley.